

Aula 1 — Fundamentos da Qualidade

"Antes de automatizar: você sabe o que testar?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Engenharia de Qualidade e Testes de Software · 24/09/2026

Bem-vindo(a) à disciplina!

Disciplina totalmente online **ao vivo**, 6 encontros síncronos de 3h30 (quintas, 19h-22h30) + trabalho assíncrono guiado entre aulas.

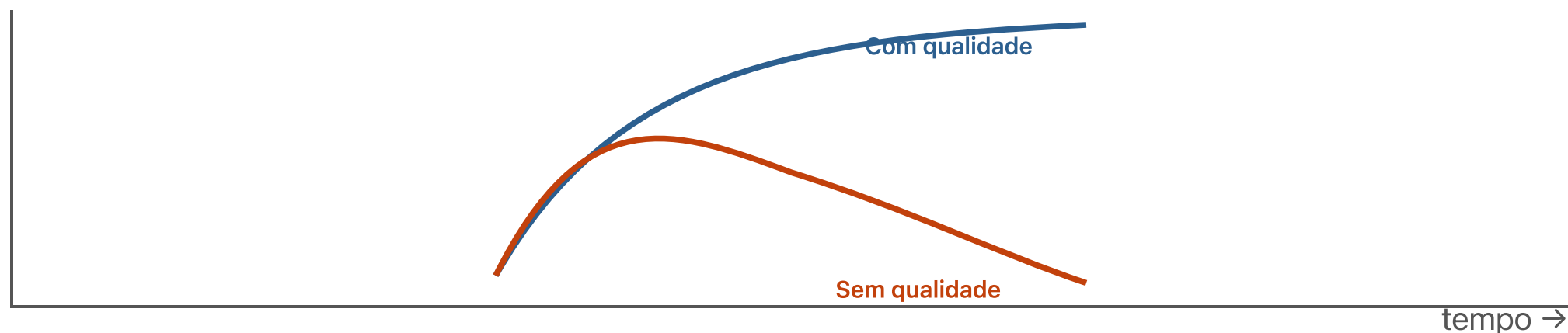
Estrutura:

- 6 aulas × 3h30 cada = 21h ao vivo + 3h assíncrono guiado = 24h totais
- Leituras dirigidas e labs entre um encontro e outro
- Office hours assíncrono via e-mail e fórum Canvas

| Bom momento pra começar: agora.

Por que investir em qualidade?

Design Stamina Hypothesis (Martin Fowler) — sem qualidade, a velocidade de entrega **despenca** com o tempo. Com qualidade, o início é mais devagar, mas a velocidade **se mantém**.



"Design activities pay off because they make it easier to evolve the software." — [Fowler, Design Stamina Hypothesis](#)

Agenda da aula (3h30)

- **19h00–20h30** (90min) — Fundamentos: qualidade, ISO/IEC 25010, tipos de teste, pirâmides, exploratória, web/mobile por dentro, acessibilidade
- **20h30–21h00** (30min) — Intervalo
- **21h00–22h30** (90min) — Lab hands-on: testes manuais no TestesQAMobile + fechamento

Deck de hoje é denso — passamos rápido pelas tabelas de referência, com tempo garantido pro lab depois do intervalo.

Objetivos da Aula 1

- **Compreender** ISO/IEC 25010:2023 (9 características de qualidade) e sua evolução histórica
- **Classificar** tipos de teste (funcional, usabilidade, performance, etc.) e aplicar num app real
- **Diferenciar** Pirâmide de Cohn, Testing Trophy (Dodds), Honeycomb (Spotify)
- **Distinguir** web testing vs mobile testing
- **Praticar** teste exploratório com heurísticas (FEW HICCUPPS, charters)
- **Documentar** um bug encontrado num formato profissional

Quiz inicial — você concorda?

| *"Se o sistema não trava, ele já está bem testado."*

| *"Teste manual é só ficar clicando sem método."*

| *"Emulador ou navegador headless testam igual ao ambiente real do usuário."*

| *"Achei poucos bugs = o sistema está pronto."*

| *"Só quem programa consegue testar de verdade."*

Spoiler: todas falsas. Veremos por quê — e vocês vão provar isso sozinhos no lab de hoje.

A crise da qualidade — mobile

- **90%** dos usuários já pararam de usar um app por causa de mau desempenho (crash, travamento, lentidão) — *App Attention Span Study*, [AppDynamics](#) + Goldsmiths, University of London (dado de indústria — AppDynamics vende ferramenta de monitoramento de performance)
- Fragmentação agrava tudo: **milhares** de combinações OS + versão + fabricante só no Android
- 1 review de 1 estrela por bug crítico é visível **pra sempre** na loja — diferente de um bug corrigido silenciosamente na web

| Testar não é burocracia. É a diferença entre o app sobreviver na loja ou não.

A crise da qualidade — web

- **53%** dos visitantes mobile abandonam um site que demora **mais de 3s** pra carregar — [Google, The Need for Mobile Speed](#)
- Web soma sua própria fragmentação: **browsers × viewports × JS habilitado ou não**
- **Caso real:** Southwest Airlines, dez/2022 — uma tempestade de inverno bagunçou a escala de voos, e o **sistema de escala de tripulação** (software antigo, sem atualização há anos) não conseguia recalcular automaticamente onde cada piloto/comissário deveria estar. Funcionários tiveram que remontar a escala **manualmente, por telefone**, um por um — o atraso virou efeito cascata: **>15 mil voos cancelados, US\$ 825 milhões** de prejuízo — [NPR](#) (rádio pública americana, sem fins lucrativos)

ISO/IEC 25010 — uma norma, 3 gerações

ISO/IEC 9126

1991 (rev. 2001)

6 características

ISO/IEC 25010

2011 — 1ª edição

8 características (+Security,
+Compatibility)

ISO/IEC 25010

2023 — 2ª edição (atual)

9 características (+Safety)

- **9126→25010 (2011)**: entra na família **SQuaRE** (System/Software Quality Requirements and Evaluation); soma Security e Compatibility como características próprias
- **2011→2023 (2ª ed.)**: **Usability** → **Interaction Capability**, **Portability** → **Flexibility**, e entra **Safety** (não endpoint acidentes/danos)

A norma que usamos hoje é a **2023** — é a que vale citar em prova, TCC e Test Strategy Document.

ISO/IEC 25010:2023 — as 9 características

1. Functional Suitability
completude funcional

2. Performance Efficiency
eficiência de recursos

3. Compatibility
interoperabilidade

4. Interaction Capability
era "Usability"

5. Reliability
robustez, disponibilidade

6. Security
proteção contra ameaças

7. Maintainability
facilidade de mudança

8. Flexibility
era "Portability"

9. Safety
novo — não causar dano

Laranja = mudou na 2ª edição (2023). No lab de hoje: bugs de 1.1 e 2.1 caem em **Functional Suitability** e **Interaction Capability**.

Fonte: [ISO/IEC 25010:2023](#) · resumo comparativo: [arc42 Quality Model](#)

ISO 25010 em detalhe (1/3)

Característica	Definição oficial	Exemplo mobile	Exemplo web
Functional Suitability	Fornece funções que atendem necessidades declaradas e implícitas	App favorita filme e mantém após fechar	Carrinho soma o total certo
Performance Efficiency	Desempenho relativo à quantidade de recursos usados	Lista de 1000 itens rola sem travar	Página carrega em <2s
Compatibility	Troca informação com outros produtos/sistemas	Funciona em Android 10 e 14	Layout correto em Chrome e Safari

ISO 25010 em detalhe (2/3)

Característica	Definição oficial	Exemplo mobile	Exemplo web
Interaction Capability <i>(era Usability)</i>	Pode ser operado por usuários específicos pra atingir objetivos específicos	Botão "voltar" onde o usuário espera	Formulário com labels e erros claros
Reliability	Realiza funções especificadas sob condições especificadas por um período	App não crasha após 1h de uso contínuo	Site no ar durante pico (Black Friday)
Security	Protege dados garantindo acesso apropriado ao nível de autorização	Senha não fica salva em texto plano	HTTPS + proteção contra SQL injection

ISO 25010 em detalhe (3/3)

Característica	Definição oficial	Exemplo mobile	Exemplo web
Maintainability	Pode ser modificado com efetividade e eficiência pelos mantenedores	Código modular — fácil adicionar 1 feature	Componentes reutilizáveis no front
Flexibility (era <i>Portability</i>)	Pode ser adaptado pra diferentes ambientes de hardware/software	Roda bem em telas pequenas e grandes	Responsivo mobile e desktop
Safety (nova, 2023)	Evita dano a pessoas, negócio, software, propriedade ou ambiente	App médico não permite dose errada	Sistema de compra não reserva assento inexistente

As 9 completas, com subcaracterísticas: [ISO/IEC 25010:2023](#)

Tipos de teste — a classificação que vamos usar hoje

Categoria	Pergunta que responde	No lab de hoje?
Funcional	O recurso faz o que deveria?	✓ Exercício 1.1
Usabilidade	É fácil e intuitivo de usar?	✓ Exercício 2.1
Interface (UI)	Layout, alinhamento, responsividade estão corretos?	—
Performance	Responde rápido? Consome poucos recursos?	—
Compatibilidade	Funciona em diferentes OS/devices/versões?	—
Segurança	Resiste a uso malicioso ou dados sensíveis expostos?	—
Acessibilidade	Pessoas com deficiência conseguem usar?	—
Instalação/Conectividade	Instala certo? Se comporta bem offline/rede ruim?	—

Cada bug tem uma categoria. Categorizar é o primeiro passo pra saber **onde** olhar e **o que** priorizar. Hoje focamos em 2 das 8 — as outras 6 aparecem nos exercícios extras de casa.

Como testar cada tipo?

- **Com roteiro (scripted):** casos de teste pré-definidos, passo a passo, resultado esperado conhecido. Bom pra regressão e cobertura previsível.
- **Sem roteiro (exploratório):** você define o objetivo, mas descobre os passos testando ao vivo — precisa de **método**, senão vira só clicar sem rumo.

Hoje em aula: 2 exercícios **com roteiro**. Em casa: exercícios **sem roteiro**, usando o que vem a seguir.

Como saber que é bug? — FEW HICCUPPS (Bolton/Bach)

Sem roteiro, ninguém te diz o "resultado esperado". Essa heurística é uma lista de perguntas — um **oráculo** — pra decidir se algo é falha:

Letra	Pergunta
Familiar	É consistente com problemas que já vimos antes?
Explicável	Dá pra explicar o comportamento observado?
World (Mundo)	É consistente com o que sabemos do mundo real?
História	É consistente com versões anteriores do produto?
Imagem	É consistente com a imagem que a empresa quer passar?
Comparáveis	É consistente com produtos parecidos?
Claims	É consistente com o que documentação/propaganda promete?
Usuários	É consistente com o que o usuário espera?
Produto	É consistente com o resto do próprio produto?
Definição	É consistente com a definição formal do produto?

Sua missão — Charter (SBTM)

Session-Based Test Management (Bach): sessão de teste **cronometrada**, guiada por uma missão curta, não por um script.

Template: *"Explorar [área/feature] procurando problemas de [tipo de teste], usando [dados/condições específicas]"*

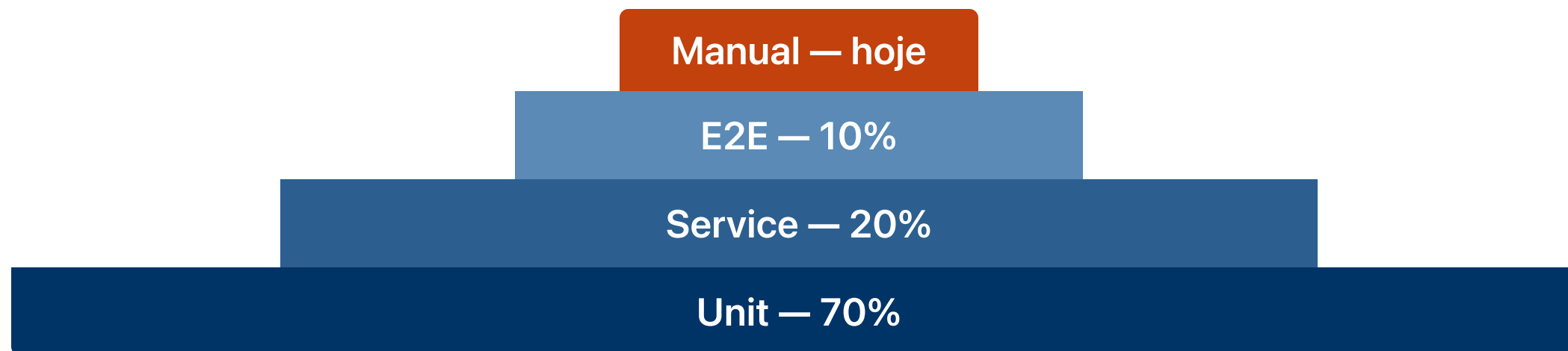
Exemplo: *"Explorar a Calculadora procurando problemas Funcionais, usando números negativos, decimais longos e divisão por zero."*

Como rodar (20-30min):

1. Escreva o charter (1 frase)
2. Explore cronometrado, anotando o que testou
3. Ao achar algo estranho, pergunte: **isso bate com FEW HICCUPPS?**
4. Registre os bugs confirmados no formulário (Canvas)

Pirâmide de testes (Cohn) — clássica web

As camadas abaixo são todas **automatizadas** — conteúdo da Aula 2. Hoje fizemos só o topo: **manual/exploratório**. É pequeno, mas é onde toda pirâmide começa na prática.

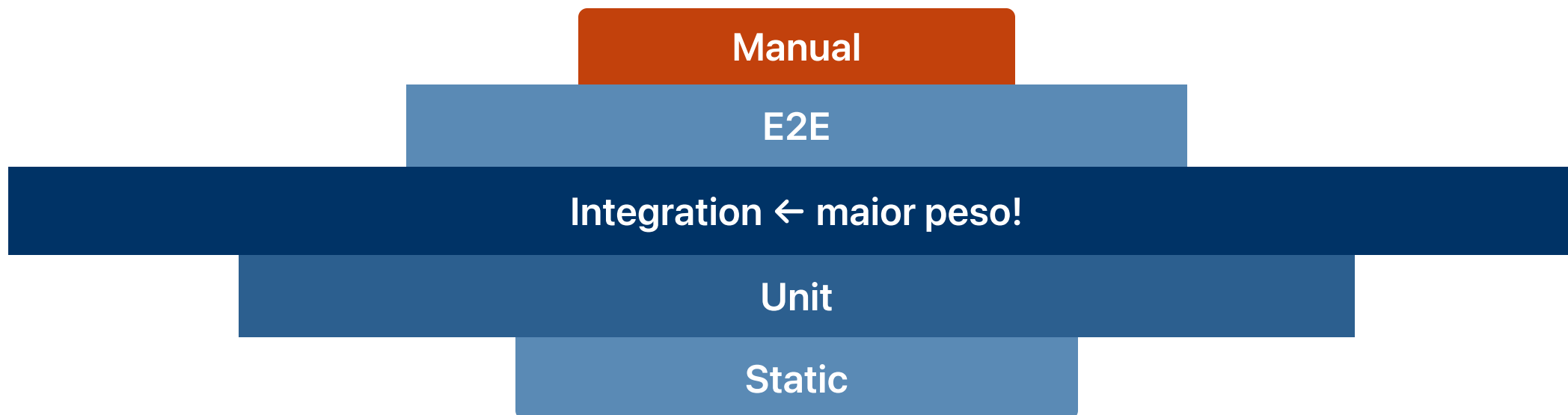


Por que pirâmide? E2E é caro, lento, frágil. Unit é barato, rápido, focado.

Aplica-se bem em projetos com bom design (interfaces estáveis, dependências claras).

Testing Trophy — Kent C. Dodds

Alternativa para apps JavaScript modernos:

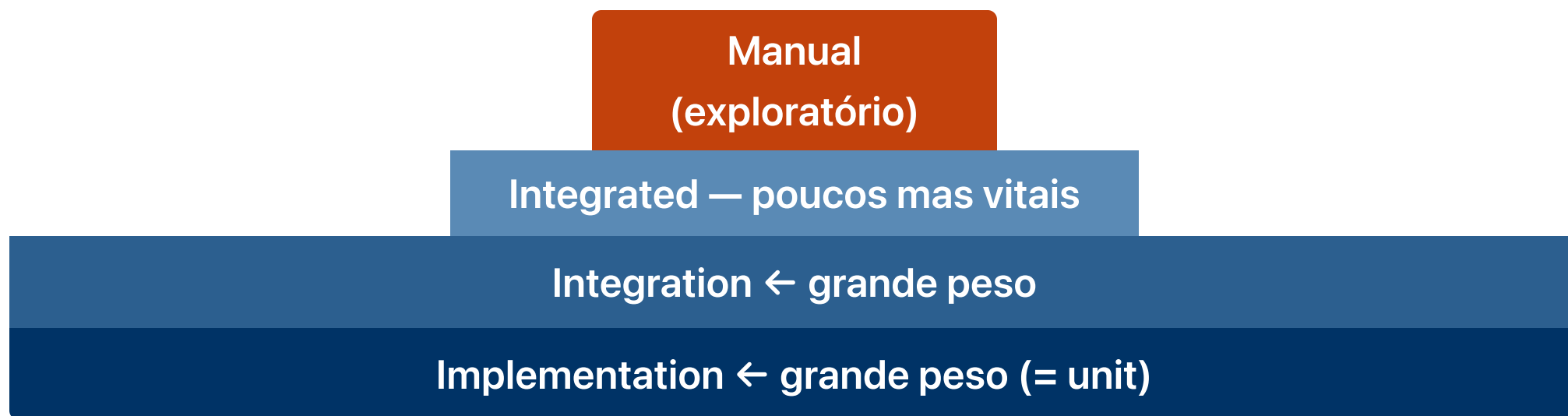


Argumento: integration tests dão mais confidence per dollar em apps com muitos mocks de unit.

Não é "pirâmide errada" — é trade-off contextual.

Honeycomb — Spotify

Para microsserviços:



Spotify defende que microsserviços precisam de balanço unit + integration; não de muitos E2E.

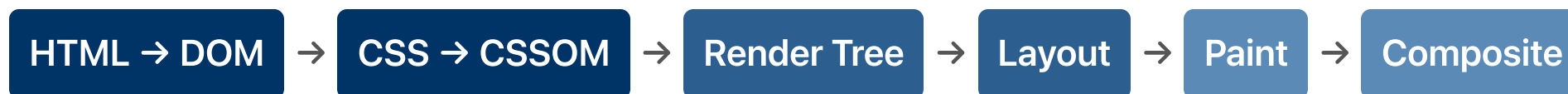
Quando usar qual?

Estratégia	Bom para
Pirâmide Cohn	Sistemas com interfaces estáveis, equipes pequenas
Testing Trophy	Apps web/RN com data fetching + state management
Honeycomb	Microserviços, equipes distribuídas

Não há resposta universal. **Contexto define.** Mas em **todas** as 3, o manual/exploratório (laranja) vem antes — é ele que te diz **o que** vale a pena automatizar depois.

Como um app web funciona por dentro

O browser recebe HTML/CSS/JS e monta a tela em etapas — cada etapa é um lugar onde qualidade pode quebrar:



- **DOM** = estrutura do HTML · **CSSOM** = estilos calculados · **Render Tree** = os dois combinados (só o que é visível)
- **JS** pode rodar no meio do processo e **mudar tudo de novo** (re-render)

Isso não existe no mobile nativo — é específico de como browsers funcionam.

Por que a página "trava"?

- **CSS bloqueia** — o browser não pinta nada até calcular o CSSOM inteiro (por isso CSS vai no `<head>`)
- **JS sem `async` / `defer` bloqueia o parsing do HTML** — o browser para de ler a página pra rodar o script
- **Imagens NÃO bloqueiam a renderização** — a página aparece sem elas e as imagens "aparecem depois"... só que se a imagem não tem `width / height` reservado, o layout **pula** quando ela chega (Cumulative Layout Shift — métrica real do Google, Core Web Vitals)

O mito não é "trava até baixar imagem" — é "**layout pula quando a imagem carrega sem espaço reservado**". Testável, mensurável, e é exatamente o tipo de bug de Performance Efficiency que testamos.

Clientes da web = motores de renderização

Todo browser implementa o **mesmo padrão** (HTML/CSS/JS), mas cada um com seu próprio motor:

Motor	Browsers
Blink	Chrome, Edge, Opera, Brave
Gecko	Firefox
WebKit	Safari (obrigatório em todo browser iOS, até Chrome no iPhone)

Pequenas diferenças de implementação = bugs de **Compatibility** reais. É por isso que "funciona no Chrome" não é sinônimo de "funciona".

Como um app mobile roda por dentro

Não existe "recarregar a página". O app abre uma vez e fica **vivo na memória** até ser fechado:

- **Nativo** (Swift/UIKit, Kotlin/Jetpack Compose): código desenha direto nos widgets do sistema operacional
- **Cross-platform** (React Native, Flutter): sua UI vira uma "ponte" que traduz pra widgets nativos (RN) ou desenha tudo sozinho num canvas próprio (Flutter/Skia)

Consequência pra QA: estado que "vazou" de uma tela pra outra é bug comum no mobile e quase não existe na web (onde F5 zera tudo).

Lifecycle mobile — o app nunca começa do zero



- Ligação, notificação, trocar de app → força ida pro **Background** — o SO pode **matar o app** pra liberar memória a qualquer momento
- Quando o usuário volta, o app deveria **restaurar o estado** exatamente onde parou — muitos bugs de qualidade nascem exatamente aqui

| Testamos isso na Aula 4 (Maestro) simulando interrupções de verdade.

Onde cada plataforma quebra

Web	Mobile
CSS/JS render-blocking	App morto em background perde estado
Imagem sem tamanho → layout shift	Permissão (câmera/local) interrompe o fluxo no meio
Cache/cookies desatualizados	Fragmentação de OS (versão antiga sem recurso novo)
JS quebra silenciosamente (<code>console.error</code> ninguém vê)	Memória limitada — app trava em listas grandes

Cada linha dessa tabela é um tipo de teste que já vimos hoje (Performance, Compatibility, Funcional...).

Acessibilidade — por que é requisito de qualidade

- **16% da população mundial** (1,3 bilhão de pessoas) vive com alguma deficiência significativa — [OMS](#)
- No Brasil, a **Lei Brasileira de Inclusão** (Lei 13.146/2015) **exige** que sites de empresas com sede/representação no Brasil sejam acessíveis — [Planalto](#)
- Não é feature extra: a ISO/IEC 25010:2023 incluiu **inclusivity** como subcaracterística de **Interaction Capability**

| Se seu app não é acessível, ele reprova qualidade — não é "nice to have".

Acessibilidade na prática

	Web	Mobile
Leitor de tela	NVDA, VoiceOver (Safari)	TalkBack (Android), VoiceOver (iOS)
Estrutura	HTML semântico (<button> , não <div onclick>) + ARIA	Labels de acessibilidade nos componentes nativos
Contraste	WCAG: mínimo 4.5:1 texto normal	Mesmo princípio, testável com ferramentas do SO
Tamanho de toque	—	Mínimo 44×44pt (iOS) / 48×48dp (Android)
Texto escalável	Zoom do browser não pode quebrar layout	Dynamic Type / fonte do sistema

| Testável hoje mesmo: ativem o leitor de tela do celular e tentem usar o TestesQAMobile.

O que o usuário espera — heurísticas de Nielsen

10 heurísticas de usabilidade (Jakob Nielsen, 1994, ainda usadas em 2026) — o que o usuário espera de qualquer interface:

1. Visibilidade do status do sistema
2. Compatibilidade com o mundo real
3. Controle e liberdade do usuário
4. Consistência e padrões
5. Prevenção de erros
6. Reconhecer em vez de lembrar
7. Flexibilidade e eficiência de uso
8. Design estético e minimalista
9. Ajudar a reconhecer e recuperar de erros
10. Ajuda e documentação

Web testing vs Mobile testing

Aspecto	Web	Mobile
Plataformas	Chromium, Firefox, WebKit	iOS + Android (+ minoritárias)
Atualização	Refresh	Build + store
Fragmentação	Browsers + viewports	OS + devices + sensores
Sensores	Limitado	Câmera, GPS, biometria, NFC
Permissões	Cookies/storage	+ camera, location, contacts, push
Background	Limitado	Lifecycle complexo (foreground/background)

Tempo desta disciplina: **1 aula fundamentos + 2 aulas web + 1 aula mobile + 1 aula IA + 1 aula CI/CD e Projeto Final** (6 aulas no total).

Automação existe — mas não é hoje

Boa parte do que testamos manualmente hoje **pode** virar teste automatizado — código que roda sozinho, sem clique humano, toda vez que o app muda.

Hoje ainda não escrevemos automação: primeiro aprendemos **o que** testar manualmente, pra depois saber **o que vale a pena** automatizar.

Regra prática: só automatize um teste que você já executou manualmente e entende exatamente o que ele verifica.

Onde rodar o teste — mobile

Ambiente	Prós	Contras
Device físico	Comportamento real (sensores, performance, rede)	Você só tem 1-2 modelos
Emulador/Simulador	Grátis, rápido de resetar, várias versões de OS	Não pega bugs de hardware real (câmera, GPS real)
Cloud device farm (BrowserStack, Firebase Test Lab)	Matriz real de devices/OS sem comprar todos	Custo, tem free tier limitado

O app de hoje: testem no **seu próprio celular** (device físico) — é o ambiente mais realista que vocês têm à mão agora.

Onde rodar o teste — web

Ambiente	Prós	Contras
Browser real (sua máquina)	Comportamento real de renderização/rede	Só o browser/OS que você tem instalado
Headless (Chromium/Firefox/WebKit local)	Grátis, rápido, roda em CI	Não pega tudo que um browser real com GPU pega
Cloud grid (BrowserStack, Sauce Labs, LambdaTest)	Matriz real de browser x versão x OS sem manter tudo local	Custo, tem free tier limitado

Mesma lógica do mobile: comece no ambiente que você já tem (seu navegador), a matriz completa vem depois.

Como documentar um bug (formato profissional)

Campo	Exemplo
ID	BUG-001
Tipo	Funcional
Pré-condição	App aberto, tela de Cadastro
Passos	1. Digitar "a@" no campo email · 2. Tocar em Salvar
Resultado esperado	Erro de validação, não salva
Resultado obtido	Salva normalmente, sem erro

Sem isso, um bug reportado vira "não funciona" — inútil pra quem vai corrigir. É o formato que vocês usam no Passo 3 do lab de hoje.

Test Strategy Document

Documento que comunica estratégia de teste:

1. **Escopo** — o que testa, o que não testa
2. **Tipos de teste** — pirâmide aplicada
3. **Ferramentas** — stack escolhida com justificativa
4. **Ambientes** — dev, staging, prod, devices
5. **Métricas** — flake rate, escape rate, coverage
6. **Responsabilidades** — quem escreve, quem mantém
7. **CI/CD** — gates, sharding

| Documento vivo, revisado a cada release major.

O app de hoje: Testes QA Mobile

Testes QA Mobile



App educacional publicado nas duas lojas — código-fonte privado, testamos de fora como QA real.

 [Google Play](#)

 [App Store](#)

Instalem agora, antes do intervalo — assim já chegam prontos pro lab.

Lab hands-on (75min) — testes manuais por tipo

Sem código hoje. App real publicado (App Store/Play Store), bugs propositalmente organizados por categoria — aplica direto os "tipos de teste" que acabamos de ver.

Roteiro

1. Abrir o app TestesQAMobile (App Store / Play Store) (10min)
2. Exercício 1.1 — Cadastro de Usuário (Funcional): roteiro + casos já no app, achar 3 bugs (25min)
3. Exercício 2.1 — Onboarding (Usabilidade): mesma mecânica, achar 3 bugs (25min)
4. Documentar 1 bug de cada no formato ID/Tipo/Passos/Resultado (10min)
5. Fechamento: teste manual é qual fatia da pirâmide? (5min)

Roteiro: `exercicios/00-warmup-aula1-tipos-de-teste/` · App publicado, código privado — testamos de fora, como QA real.

Leituras obrigatórias

- Crispin, L.; Gregory, J. — *Holistic Testing* (Pearson 2024), cap 1–3
- Bolton, M. — *FEW HICCUPPS* (heurística de oráculo, usada no lab de hoje)
- Bach, J. — *Session-Based Test Management*
- Dodds, K. — *Write Tests. Not Too Many. Mostly Integration.*
- ISO/IEC 25010:2023 — resumo guiado (link no Canvas)
- Spotify Engineering — *Testing of Microservices* (Honeycomb)

Bibliografia completa da disciplina: [BIBLIOGRAFIA.md](#)

Próxima aula — Aula 2 (01/10)

Playwright Avançado — começamos a automatizar o que hoje foi manual

- Page Object Model + Screenplay Pattern
- Flakiness: métricas, detecção, mitigação
- Locators recomendados
- Network mocking + HAR replay
- storageState (auth reuse)
- Visual regression + sharding 4-way em CI

Bons estudos!

✉ jackson.96@gmail.com (também canal Canvas)

🔗 linkedin.com/in/3jacksonsmith

🐙 github.com/jacksonsmith

Office hours: quintas 19h via Google Meet (link Canvas)